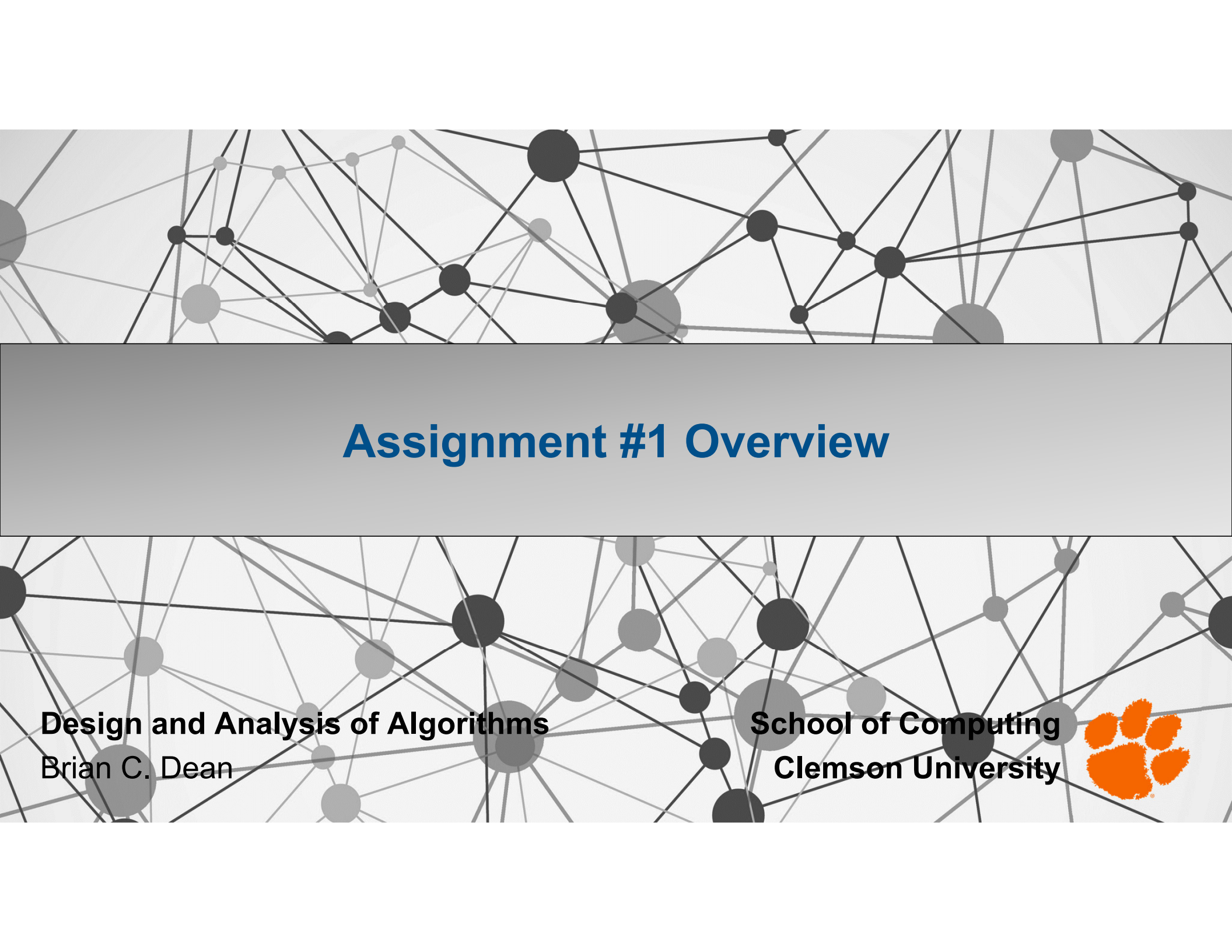


The background of the slide features a complex network diagram. It consists of numerous circular nodes of varying sizes, some in dark gray and others in light gray, interconnected by a web of thin, dark gray lines. The nodes are distributed across the entire slide, with a higher density in the upper and lower portions, framing the central title area.

Assignment #1 Overview

Design and Analysis of Algorithms
Brian C. Dean

School of Computing
Clemson University



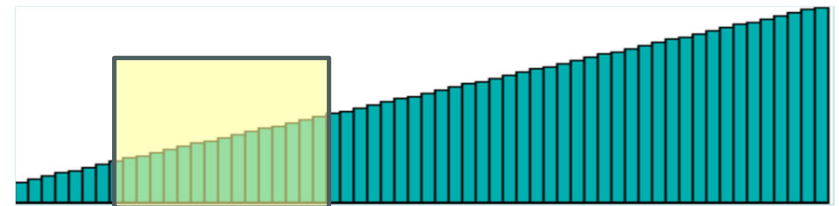
Coding Assignments

- General goals:
 - Reinforce lecture content.
 - Showcase prominent use cases relevant to modern practice.
 - Improve our implementation skills.
- Automatic grading with feedback on performance.
- Usually 2..3 weeks time to complete each one.
 - Some assignments (coding, algorithm design) may overlap in their time ranges slightly, to allow more generous due dates.

Part A: Warm-Up

Write Binary Search

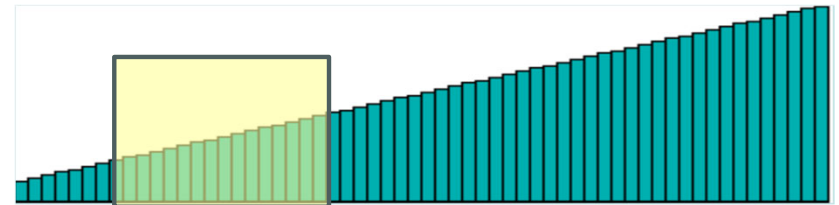
- You'll be given `hw1a_soln(.cpp, .java, .py)` to modify.
- Submit that one file (can add code anywhere you want in it).
- Goal: In an integer array $A[0 \dots N-1]$, find index i with $A[i]$ being closest to a given target integer T .
- `call_grader(i)` to get back the value of $A[i]$. ≤ 32 calls max!
- When you are done, print the value of i that is your answer.



Part A: Warm-Up

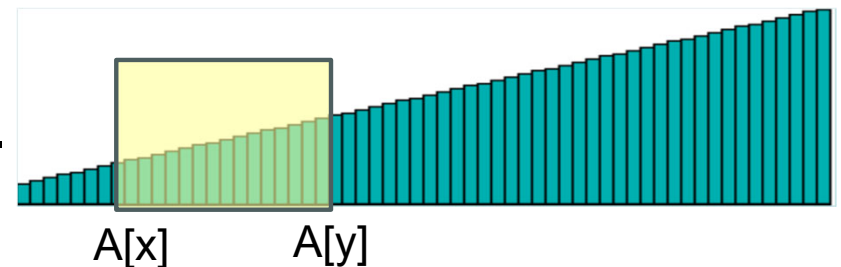
Write Binary Search

- To test locally, use `call_grader_local(i)` instead of `call_grader(i)`.
 - This uses a simple 10-element array with values 1000, 2000, 3000, ..., 9000, 10,000 with target $T = 8400$.
- 10 total test cases (case 1, not counted, same as above).
- Score on each case 100 or 0 based on correctness.
- Array size N and target T passed as environment vars.



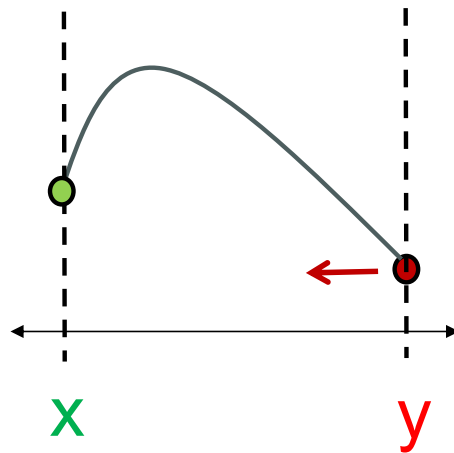
Assignment #1: Extending Binary Search for Model Parameter Tuning

- Binary search has many interesting extensions...
 - Interpolation search: $O(\log \log N)$ expected time, if elements in our array come from a known probability distribution
 - Approximate: narrow to index range $[x, y]$ with error $y / x \leq 1.01$.
 - Can do this also in $O(\log \log N)$ time also using *geometric mean* $\sqrt{xy}$ instead of arithmetic mean $(x+y) / 2$ for next guess (since this reduces y / x to $\sqrt{y / x}$ at each step).
 - Asymmetric guessing penalties: e.g., \$1 if too low, \$99 if too high.
 - Unbounded: “I’m thinking of a positive integer...”

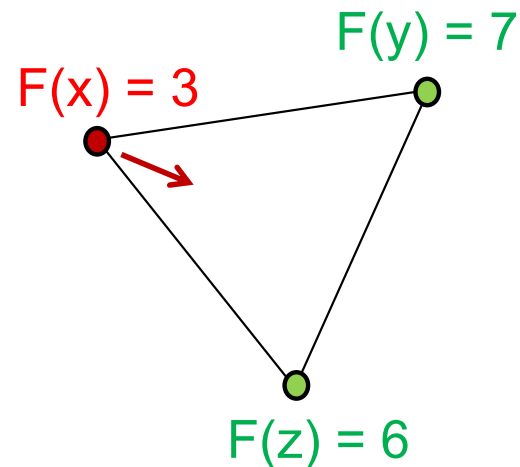


“Binary Search” in Optimization

The “Nelder-Mead” / simplex search / amoeba search algorithm:

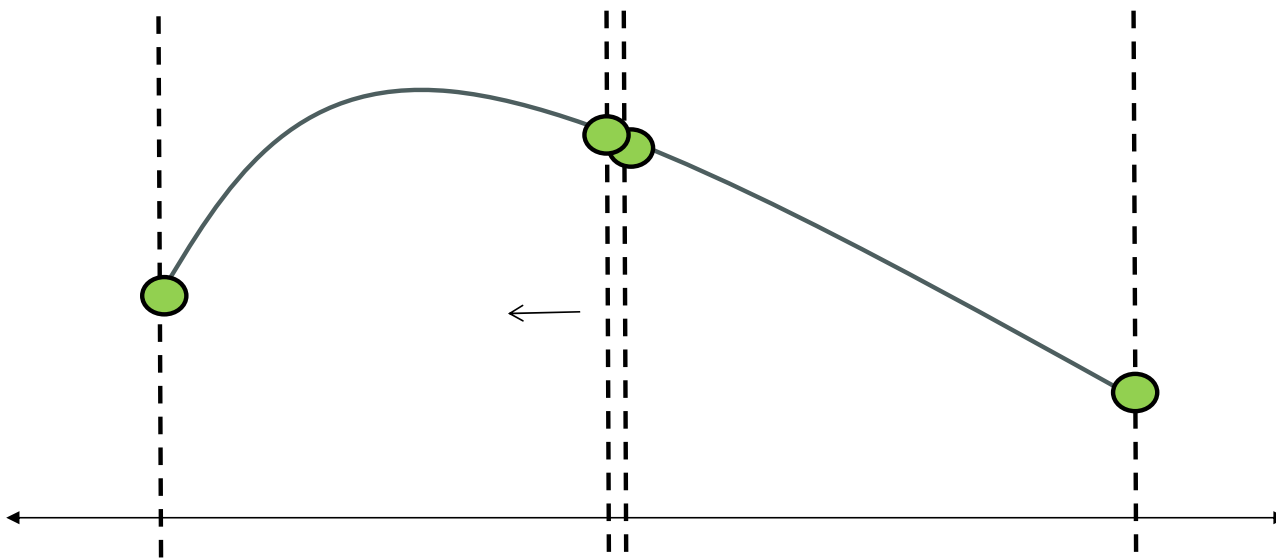


In d dimensions, “bracket”
solution with $d+1$ points



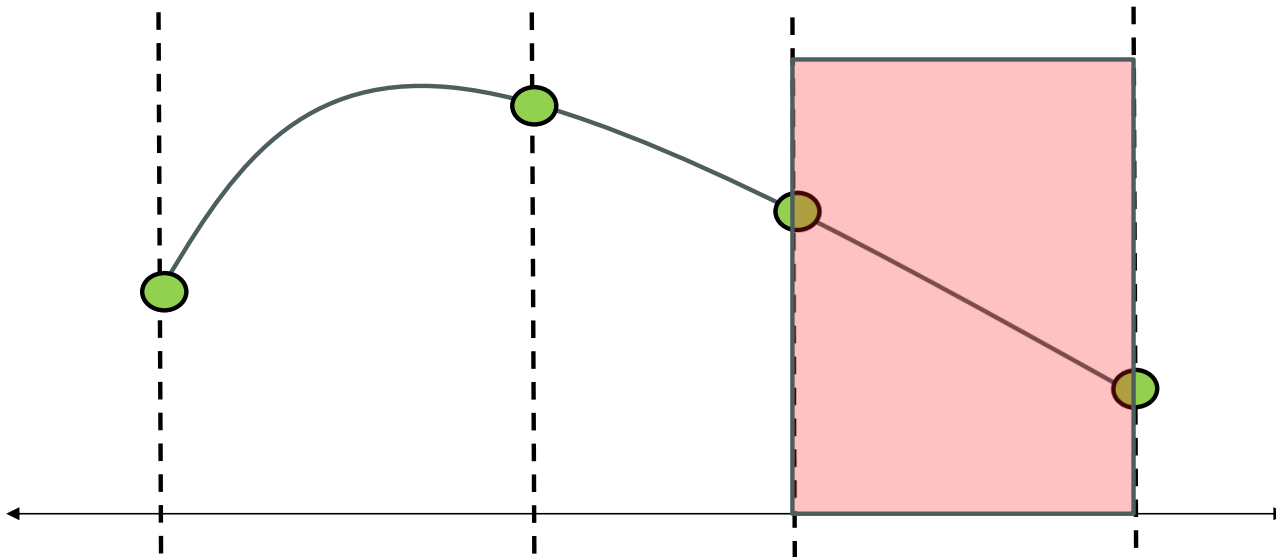
Move “bracket” with
worst function value in
direction of others

Focus on 1 Dimension and a Unimodal Function



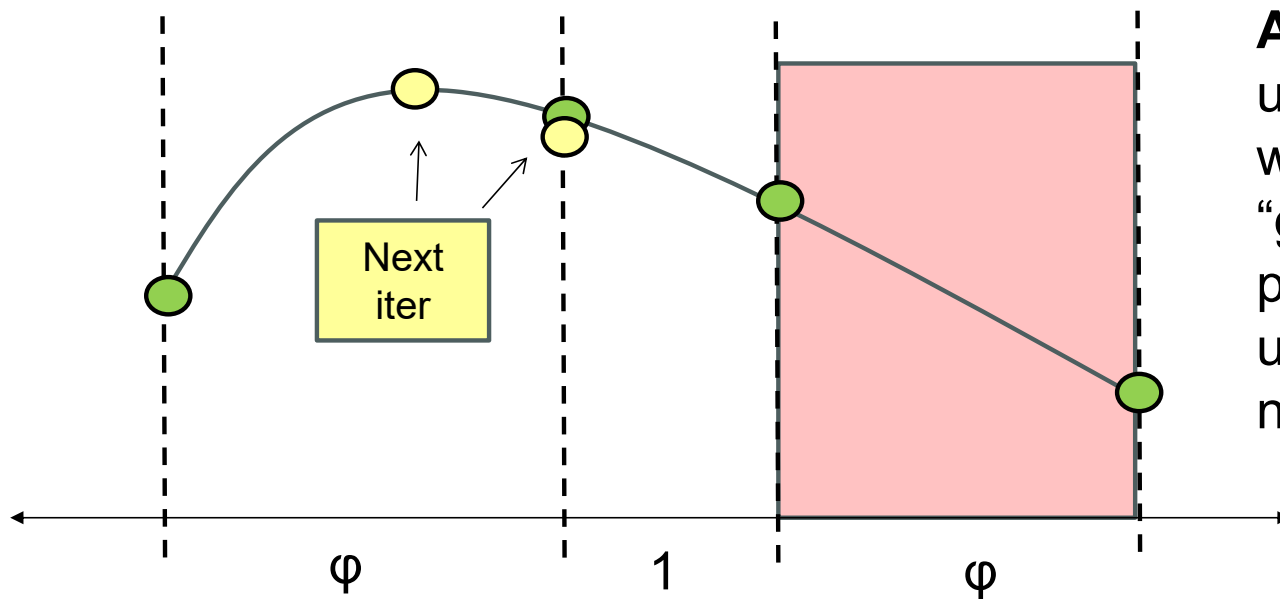
Could evaluate twice at very nearby points in middle of range, to learn direction function is increasing (i.e., estimate derivative / slope at midpoint).

Ternary Search



Evaluate at any two points, dividing current interval into three pieces.
By looking at the four resulting values, can exclude either 1st or 3rd piece.

Ternary Search

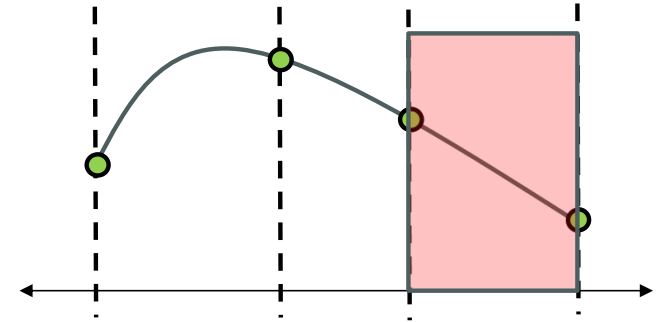


Aside: Golden section search uses inter-point spacing $\varphi:1:\varphi$ with $\varphi = (1+\sqrt{5}) / 2$ being the “golden ratio”. Makes an eval point in current iteration line up with an eval point in the next iteration, saving evals.

Evaluate at any two points, dividing current interval into three pieces.
By looking at the four resulting values, can exclude either 1st or 3rd piece.

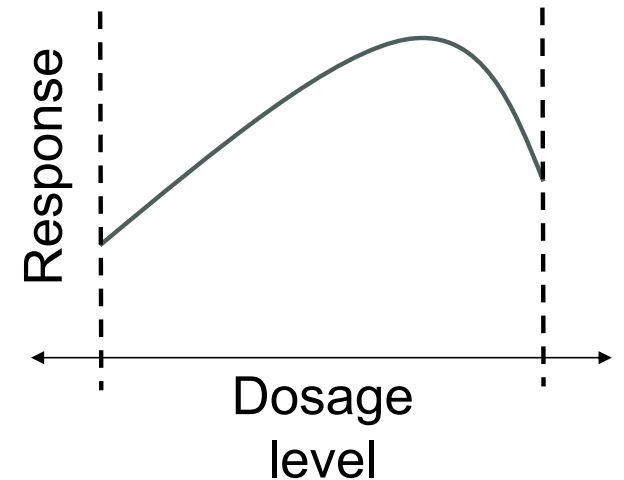
Assignment Part B: “Ternary Search” / Finding Max in a Unimodal Array

- In $A[0 \dots N-1]$, find i maximizing $A[i]$.
- Initial code: `hw1b_soln(.cpp, .java, .py)`
- Similar setup as part A.
- Invoke `call_grader(i)` at most 64 times.
- Use `call_grader_local(i)` for local testing on a simple case.
- 10 test cases (first doesn't count; same as local example).
- Score on each case 100 if correct, 0 if not.



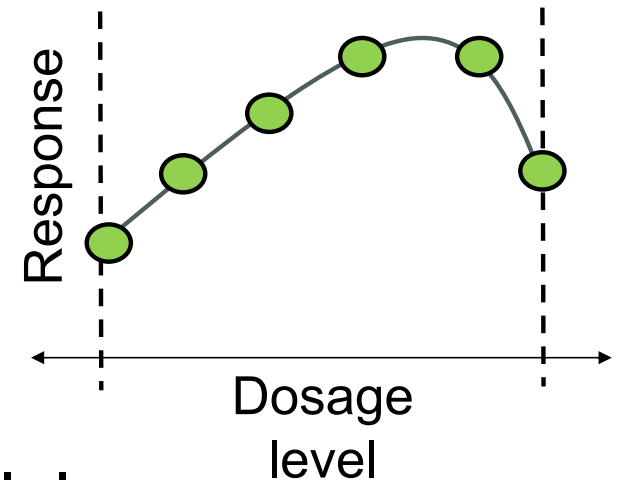
Motivation for this Assignment: Parameter Fitting

- Trying to find optimal setting for some key model parameter leading to unimodal behavior.
- However, expensive to evaluate model to test out parameter values.



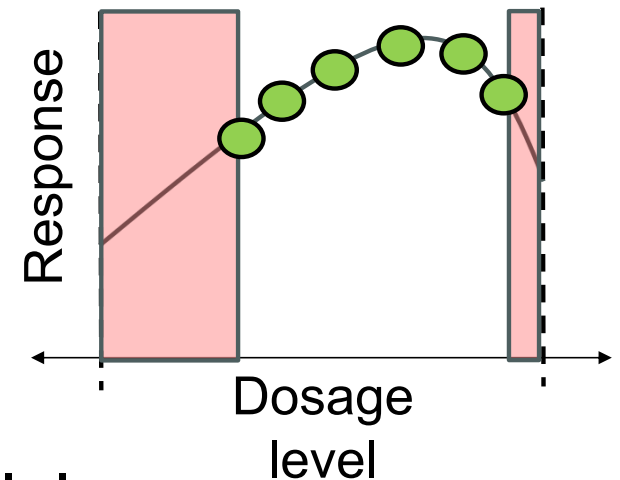
Motivation for this Assignment: Parameter Fitting

- Trying to find optimal setting for some key model parameter leading to unimodal behavior.
- However, expensive to evaluate model to test out parameter values.
- Obvious idea: test K parameter choices at once, running models in parallel.



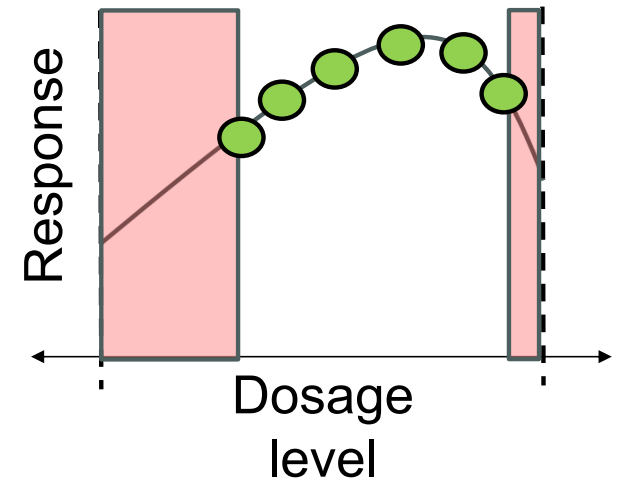
Motivation for this Assignment: Parameter Fitting

- Trying to find optimal setting for some key model parameter leading to unimodal behavior.
- However, expensive to evaluate model to test out parameter values.
- Obvious idea: test K parameter choices at once, running models in parallel.
... and do this repeatedly for R rounds.



Assignment Part C: Parallel Multi-Round “Binary Search”

- Goal: find x in $[0, 1]$ maximizing $f(x)$.
- In each of R rounds, use `call_grader()` to probe $f(x)$ for K values of x .
- K and R provided as environment vars.
- `call_grader_local()` for local testing.



Curveball: Noisy Function Evals!

- Goal: find x in $[0, 1]$ maximizing $f(x)$.
- In each of R rounds, use `call_grader()` to probe K values of x .
 - For each, you'll get $f(x) \pm \text{some noise}$.
(noise different each time $f(x)$ evaluated)
- Score (up to 100) based on how close your answer is to the optimal x .
- 20 test cases
 - Cases 1-5 no noise. 5-10 low noise. 11-15 medium. 16-20 high

